Your second attempt centered around *Ghost Methods (57)* (to be more precise, it used a *Dynamic Proxy (60)* that is also a *Blank Slate (66)*):

methods/computer/blank_slate.rb

```
class Computer < BasicObject
  def initialize(computer_id, data_source)
    @id = computer_id
    @data_source = data_source
  end

  def method_missing(name, *args)
    super if !@data_source.respond_to?("get_#{name}_info")
    info = @data_source.send("get_#{name}_info", @id)
    price = @data_source.send("get_#{name}_price", @id)
    result = "#{name.capitalize}: #{info} ($#{price})"
    return "* #{result}" if price >= 100
    result
  end
end
```

Neither solution would be practical without Ruby's dynamic capabilities. If you come from a static language, you're probably accustomed to spotting and removing duplication *inside* your methods. In Ruby, you might want to look for duplication *among* methods as well. Then you can remove that duplication with some of the spells you've learned today.

You and Bill can consider the two solutions. It's time to make a choice. Which one do you like best?

Dynamic Methods vs. Ghost Methods

As you experienced yourself, *Ghost Methods (57)* can be dangerous. You can avoid most of their problems by following a few basic recommendations (always call `super`, always redefine `respond_to_missing?`)—but even then, Ghost Methods can sometimes cause puzzling bugs.[2]

The problems with Ghost Methods boil down to the fact that they are not really methods; instead, they're just a way to intercept method calls. Because of this, they behave differently from actual methods. For example, they don't appear in the list of names returned by `Object#methods`. In contrast, Dynamic Methods are just regular methods that happened to be defined with `define_method` instead of `def`, and they behave the same as any other method.

2. A presentation on the perils of `method_missing` is at http://www.everytalk.tv/talks/1881-Madison-Ruby-The-Revenge-of-method-missing.